

基于阿里云与Coze平台的AI视频混剪批量生成技术实现方案

一. 执行摘要与架构概述

问题陈述

本方案旨在解决一项核心业务需求：如何通过一个用户友好的Web界面，让用户上传单个源视频后，系统能通过自动化和AI辅助的方式，批量生成大量内容相似但细节各异的视频变体，并最终将生成结果反馈给用户供其下载。此举的主要目的是为了规避在社交媒体或内容分发平台上，因重复发布相同视频而导致的内容限流或账号封禁风险。

方案提议：全栈混合式架构

经过深入分析，一个完全在Coze平台内部完成所有处理的方案在技术上是不可行的。视频渲染是计算密集型任务¹，而Coze工作流中的“代码节点”存在60秒的执行超时限制²。

因此，本方案提出一种**全栈混合式架构(Full-Stack Hybrid Architecture)**。该架构不仅包含了后端的自动化处理流程，还引入了前端用户界面和配套的后端服务(Backend for Frontend, BFF)，构成一个完整的、可交付的应用。

- 前端用户界面 (Web Frontend):** 用户进行所有操作的入口，负责视频上传和结果展示。
- 后端支持服务 (BFF):** 一组轻量级的API，作为前端和后端核心服务之间的桥梁，处理如安全上传、状态查询等非核心但必要的业务。
- Coze(“大脑”):** 作为系统的中央编排器，负责生成所有视频混剪的“制作说明书”(即编辑决策列表, EDL)，而不执行实际的视频渲染³。
- 外部视频处理后端(“肌肉”):** 独立的专业服务，负责接收EDL并执行耗时的视频渲染工

作。

组件角色定义

1. **前端用户界面 (Web Frontend):** 用户通过浏览器访问的网页。
2. **后端支持服务 (BFF):** 可由**阿里云API网关 + 函数计算**实现, 提供接口供前端调用。
3. **阿里云OSS存储桶:** 分为输入桶(存放原始视频)和输出桶(存放混剪后的视频)。
4. **触发式函数计算 (Trigger FC):** 由OSS事件触发, 负责启动Coze工作流⁵。
5. **Coze工作流(大脑):** 接收到任务后, 编排生成多个EDL⁸。
6. **视频处理后端(肌肉):** 接收EDL, 渲染视频, 并将成品存入OSS输出桶。

业务流程描述

用户在前端页面上传原始视频。上传完成后, 后端自动化流程被触发。处理期间, 前端页面可轮询查询状态。任务完成后, 页面上会显示所有生成视频的下载链接列表, 用户可直接点击下载。详细的业务流程图和分步说明请参见第二章。

二. 详细的端到端实现流程

本章节将整个自动化混剪流程分解为一系列连续的步骤, 并提供一个详细的流程图, 以便于理解各个组件之间的交互和数据流动。

流程分步详解

阶段一: 用户交互与任务发起

1. **访问与上传 (用户 -> 前端):**
 - 用户在浏览器中打开前端应用页面。
 - 点击上传按钮, 从本地选择一个原始视频文件(例如 source_video.mp4)。

2. 请求安全上传凭证 (前端 -> BFF):

- 前端向后端支持服务(BFF)发起API请求, 告知需要上传一个文件。
- BFF(一个由API网关触发的函数计算)接收到请求。

3. 生成预签名URL (BFF -> OSS -> BFF):

- BFF调用阿里云OSS的SDK, 为输入存储桶(my-video-remix-input)生成一个有时效性的、安全的“预签名上传URL”。这个URL包含了临时授权, 允许持有者在规定时间内向指定的路径上传一个文件。
- BFF将这个预签名URL返回给前端。

4. 直接上传至云端 (前端 -> OSS):

- 前端拿到预签名URL后, 直接将用户选择的视频文件通过HTTP PUT请求上传到该URL。
- **关键点:** 视频文件不经过BFF服务器, 而是由用户的浏览器直接上传到OSS, 极大地减轻了BFF服务器的带宽压力和处理负载, 提升了上传效率和系统的可扩展性。

阶段二: 后端自动化处理

5. 事件触发 (OSS -> 触发式FC):

- OSS服务检测到有新文件成功上传到输入桶。
- 由于预先配置了事件通知规则, OSS立即将包含文件信息的事件消息发送给另一个专门的、由事件触发的函数计算(Trigger FC)。

6. 调用“大脑” (触发式FC -> Coze):

- Trigger FC被激活, 解析事件消息, 获取视频的URL。
- 它向已发布的Coze工作流API端点发起调用, 将视频URL和混剪参数(如生成数量)传递过去。

7. 智能编排 (Coze工作流):

- Coze工作流开始执行, 通过其内部的LLM节点、代码节点等, 为每个视频变体生成一份详细的、包含随机元素的“编辑决策列表”(EDL)JSON。
- 最终, Coze工作流通过循环节点, 将每一份EDL逐一发送给视频处理后端。

8. 执行渲染与存储 (视频后端 -> OSS):

- 视频处理后端接收到EDL后, 开始执行繁重的渲染工作。
- 每完成一个视频的渲染, 就将其上传到指定的OSS输出存储桶(my-video-remix-output)。

阶段三: 结果回显与下载

9. 状态轮询 (前端 -> BFF):

- 在视频上传完成后, 前端页面会启动一个定时器, 定期(例如每10秒)向BFF发起API请求, 查询任务处理状态。

10. 查询结果 (BFF -> OSS -> BFF):

- BFF接收到查询请求后, 会去查询OSS输出存储桶中是否存在与本次任务相关的文件。

- 当查询到文件后，BFF会为这些文件生成有时效性的“预签名下载URL”，并将这些URL列表返回给前端。

11. 展示与下载 (前端 -> 用户):

- 前端接收到包含下载链接的列表后，停止轮询。
- 在页面上动态渲染出所有已生成视频的列表，每个列表项都是一个可点击下载链接。
- 用户点击链接，即可通过预签名URL安全地下载对应的视频文件。流程闭环。

详细流程图 (Mermaid语法)

代码段

```
graph TD
    subgraph "用户界面 (Frontend)"
        A[1. 用户在网页选择视频] --> B{2. 前端向BFF请求上传URL};
        D;
        J;
        L[11. 展示可下载的视频链接];
    end

    subgraph "后端支持服务 (BFF)"
        C;
        B --> C;
        K;
        J --> K;
    end

    subgraph "核心自动化流程"
        E(5. OSS输入桶);
        D --> E;
        E -- "6. OSS事件触发" --> F;
        F -- "调用Coze API" --> G[8a. Coze工作流];
        G -- "分发EDL" --> H[8b. 视频处理后端];
        H -- "渲染并上传" --> I(OSS 输出桶);
    end
```

```
end
```

```
C --> D;
```

```
K --> L;
```

```
K -- "查询文件" --> I;
```

三. Coze工作流:编排核心

Coze工作流是整个系统的智能中枢。它的核心任务不是处理视频数据,而是生成一份或多份结构化的“编辑决策列表”(Edit Decision List, EDL)。EDL是一个JSON对象,它以轻量级、标准化的方式描述了一个视频变体的所有构成元素。这种设计将复杂的视频渲染任务与编排逻辑完全解耦,使得Coze可以在其60秒的执行时限内轻松完成任务,同时也让整个系统更加模块化,易于调试和扩展²。

节点化实现指南

一个典型的Coze混剪编排工作流将由以下节点构成⁴:

1. 开始节点 (Start Node)

此节点定义了工作流的输入参数,是整个流程的入口。

- source_video_url (字符串): 源视频在[阿里云OSS](#)存储桶中的URL。
- num_variations (整数): 希望生成的视频变体数量,例如 10。
- clip_duration_seconds (整数): 每个视频切片的时长,例如 5。
- content_theme (字符串): 用于指导AI生成文案的内容主题,例如“励志”、“科技感”或“温馨”。

2. LLM节点 (大语言模型节点)

此节点利用Coze内置的AI能力，为每个视频变体生成独特的文本内容，这是确保视频独创性的关键一环。

- **提示词工程 (Prompt Engineering):** 精心设计的提示词是获取高质量输出的关键。

- **示例提示词:**

角色

你是一位专业的短视频文案策划。

技能

- 根据给定的主题，创作出简短、有力、引人注目的文案。

- 严格按照要求的JSON格式输出内容。

任务

根据用户提供的主题“{{content_theme}}”，为{{num_variations}}个不同的视频，每个视频生成3句独立的、长度不超过15个字的短文案。

输出格式

请将所有文案组织成一个JSON数组，其中每个元素是包含3个字符串的子数组。不要包含任何解释性文字。

- 该节点利用Coze强大的LLM能力³，确保每个最终视频的文本覆盖都是独一无二的。

3. 代码节点 (Code Node)

这是核心的逻辑处理单元，通过运行Python脚本来生成所有EDL。

- **环境:** Coze代码节点支持Python 3.11.3及其标准库，如 random 和 json²。
- **脚本逻辑:**
 1. **接收输入:** 从“开始节点”和“LLM节点”获取参数值。
 2. **定义资源库:** 在代码内部硬编码或从外部知识库获取资源列表，例如：
 - music_tracks: 一个包含多个免版权背景音乐URL的列表。
 - transitions: 一个包含可用转场效果名称的列表，如 fade, wipeleft, slideright 等¹³。
 3. **计算切片总数:** 根据传入的 source_video_url 和 clip_duration_seconds，计算出源视频可以被切割成多少个片段。（注意：获取视频总时长可能需要一次预处理或作为输入参数传入）。

4. **主循环:** 启动一个循环, 迭代 num_variations 次, 为每个视频变体生成一个EDL。
5. **循环内部操作:**
 - **随机化片段顺序:** 创建一个从0到(切片总数-1)的索引列表, 并使用 random.shuffle() 将其顺序打乱。
 - **随机选择资源:** 从 music_tracks 和 transitions 列表中随机选择一个元素。
 - **分配文案:** 从LLM节点输出的文案数组中, 按顺序取出一个文案集(包含3句话)。
 - **组装EDL:** 构建一个完整的JSON对象, 包含所有混剪所需信息。
6. **输出:** 将所有生成的EDL JSON对象汇集到一个JSON数组中, 作为此节点的最终输出。

4. 循环节点 (Loop Node)

此节点接收来自“代码节点”的EDL数组, 并对数组中的每一个EDL对象进行一次迭代。

5. 插件/API节点 (Plugin/API Node)

此节点位于“循环节点”内部, 负责在每次迭代中, 将当前的EDL发送到外部视频处理后端。

- **配置:** 配置该节点向外部后端的API端点发起一个HTTP POST请求。
- **认证:** 在请求头中设置认证信息, 如 Authorization: Bearer <YOUR_BACKEND_API_KEY> ⁸。
- **请求体 (Body):** 将循环中当前迭代的单个EDL JSON对象作为请求体发送。

工作流发布为API

完成上述节点编排后, 需要将整个工作流发布为一个API服务。这会生成一个稳定、安全的URL, 供阿里云函数计算调用。具体步骤包括: 在Coze工作流编辑界面的右上角点击“发布”, 在发布配置中选择“发布为API服务”, 并记录下生成的bot_id、workflow_id以及用于认证的个人访问令牌(Personal Access Token)⁹。

四. 视频处理后端: 方案比较与选择

系统的“肌肉”部分, 即视频处理后端, 是实现最终视频渲染的关键。对此, 存在两种主流的实现路径: 使用第三方商业API(购买服务) 或自建服务(自主构建)。这是一个典型的“构建与购买”(Build vs. Buy)的决策, 其选择不仅影响开发成本和速度, 也关乎长期的运营开销和系统灵活性。一份关于某公司从AWS MediaConvert迁移到自建Go+FFmpeg服务的案例分析¹⁸, 充分说明了这两种路径在不同场景下的合理性。

方案A (推荐, 用于快速部署): 托管式视频编辑API

- **代表服务:** Shotstack, AWS Elemental MediaConvert
- **基本原理:** 此类服务将视频渲染的复杂性完全封装, 包括服务器管理、任务队列、编解码器兼容性、弹性伸缩和存储等。用户只需通过简单的API调用, 即可完成视频生成任务¹⁹。这使得开发团队可以将精力完全集中在Coze工作流的创意逻辑构建上。
- **API交互模型 (以Shotstack为例):** Coze生成的EDL需要遵循目标服务的JSON格式。一个简化的Shotstack请求体可能如下所示²⁰:

```
JSON
{
  "timeline": {
    "tracks": [
      {
        "clips": [
          { "asset": { "type": "video", "src": "oss://my-bucket/clip1.mp4" }, "start": 0, "length": 5 },
          { "asset": { "type": "video", "src": "oss://my-bucket/clip2.mp4" }, "start": 5, "length": 5 }
        ]
      }
    ]
  },
  "output": {
    "format": "mp4",
    "resolution": "hd"
  }
}
```


方案B (高级, 用于大规模成本优化): 自建FFmpeg服务

- **架构蓝图:**

- **API层:** 使用轻量级Web框架(如Python的Flask或FastAPI)创建一个API端点, 用于接收来自Coze的EDL JSON。
- **任务执行层:** API端点接收到请求后, 根据EDL内容动态构建一个复杂的FFmpeg命令行指令, 并通过子进程执行它。
- **部署:** 将整个服务打包成Docker容器, 部署在可伸缩的云平台上, 如**阿里云容器服务ACK/ASK**或配置了自动伸缩组的ECS实例, 以应对并发的渲染请求。

- **核心FFmpeg指令集:** 这是自建方案的技术核心, 需要精确构建指令。

- **高效切片:** 使用 segment muxer可以在不重新编码的情况下快速切片, 极大提升效率。

Bash

```
ffmpeg -i source.mp4 -c copy -map 0 -segment_time 5 -f segment  
-reset_timestamps 1 clip%03d.mp4
```

- **带转场的拼接:** 利用强大的 xfade 滤镜实现平滑的片段过渡¹³。这需要构建一个复杂的滤镜图(filter graph), 并精确计算每个转场的 offset(开始时间) 和 duration(持续时间)²³。

Bash

```
ffmpeg -i clip1.mp4 -i clip2.mp4 -i clip3.mp4 -filter_complex \  
"[0:v][1:v]xfade=transition=fade:duration=1:offset=4[v1]; \  
[v1][2:v]xfade=transition=wipeleft:duration=0.5:offset=8.5[v2]" \  
-map "[v2]" output.mp4
```

- **文本叠加:** 使用 drawtext 滤镜将Coze LLM生成的文案在指定时间、位置和样式下绘制到视频上。
- **音频混合:** 使用 amix 或 amerge 滤镜将背景音乐与(可能存在的)原视频音轨混合。

视频后端方案对比

为了帮助做出明智的架构决策，下表对两种方案进行了多维度比较：

评估维度	方案A: 托管式API (如 Shotstack)	方案B: 自建FFmpeg服务
开发投入	低 (主要工作是API集成)	高 (需后端开发、DevOps、容器化知识)
上线速度	快	慢
可伸缩性	极高 (由服务商保障)	高 (但需自行设计和配置)
成本模型	按使用量付费 (如每分钟渲染时长)	按基础设施资源付费 (如vCPU/小时)
灵活性/控制力	受限于API功能集	极高, 可完全控制FFmpeg所有参数
维护成本	无	高 (需处理软件更新、安全、监控)

五. 端到端实现与代码参考

本章节提供将所有组件串联起来的关键代码和配置。

阿里云OSS与函数计算触发器设置

- 创建OSS存储桶:** 在阿里云控制台创建两个OSS存储桶，例如 my-video-remix-input 和 my-video-remix-output。
- 创建函数计算服务与函数:**
 - 登录函数计算控制台，创建一个服务。
 - 在该服务下，创建一个新的事件函数，选择合适的运行时(例如Python 3.9)。
- 配置OSS触发器:**
 - 在函数计算控制台，进入您创建的函数详情页，点击“触发器”标签页，然后“创建触发器”。
 - 触发器类型:** 选择 “对象存储OSS (Object Storage Service)”。
 - 存储桶名称:** 选择您创建的输入桶 my-video-remix-input。

- **触发事件:** 选择 `oss:ObjectCreated:PutObject`。这表示当有文件通过PutObject操作上传时触发函数⁷。
- **触发规则 (可选但推荐):**
 - **前缀:** 设置一个前缀, 如 `uploads/`, 这样只有上传到该目录的文件才会触发函数。
 - **后缀:** 设置一个后缀, 如 `.mp4`, 确保只有视频文件能触发。
- **避免无限循环:** 确保函数处理后的输出文件保存在不同的存储桶 (`my-video-remix-output`) 或不同的前缀下, 以防输出操作再次触发函数²⁵。

阿里云函数计算代码 (Python)

以下是部署在阿里云函数计算上的Python 3.x函数代码, 它负责接收OSS事件并调用Coze工作流。

Python

```
import json
import os
import requests
import urllib.parse

# 从环境变量中获取Coze API的配置信息
COZE_API_URL = os.environ.get('COZE_API_URL') # 示例:
https://api.coze.com/v1/workflow/stream_run
COZE_BOT_ID = os.environ.get('COZE_BOT_ID')
COZE_WORKFLOW_ID = os.environ.get('COZE_WORKFLOW_ID')
COZE_API_TOKEN = os.environ.get('COZE_API_TOKEN')

def handler(event, context):
    """
    此函数由阿里云OSS PutObject事件触发, 调用Coze工作流API。
    """
    try:
        # 1. 解析OSS事件, 获取存储桶和对象键名
        # OSS事件通常是bytes类型, 需要先解码
```

```

event_data = json.loads(event.decode('utf-8'))
# 根据阿里云函数计算的OSS事件结构解析 [26]
record = event_data['events']
bucket_name = record['oss']['bucket']['name']
object_key = urllib.parse.unquote_plus(record['oss']['object']['key'], encoding='utf-8')
region = record['region']

# 2. 构建源视频的OSS URL
# 格式: https://<BucketName>.<Endpoint>/<ObjectName>
source_video_url = f"https://{bucket_name}.oss-{region}.aliyuncs.com/{object_key}"

print(f"New video uploaded: {source_video_url}")

# 3. 准备调用Coze工作流API的参数
headers = {
    'Authorization': f'Bearer {COZE_API_TOKEN}',
    'Content-Type': 'application/json'
}

payload = {
    "bot_id": COZE_BOT_ID,
    "workflow_id": COZE_WORKFLOW_ID,
    "user_id": "fc_trigger_user", # 自定义一个用户ID
    "stream": False, # 使用非流式响应, 一次性获取结果
    "input": {
        "source_video_url": source_video_url,
        "num_variations": 10, # 可配置
        "clip_duration_seconds": 5, # 可配置
        "content_theme": "inspirational" # 可配置
    }
}

# 4. 发起API请求
print("Invoking Coze workflow...")
response = requests.post(COZE_API_URL, headers=headers, json=payload,
timeout=30)
response.raise_for_status() # 如果HTTP状态码不是2xx, 则抛出异常

print(f"Coze workflow invoked successfully. Status: {response.status_code}")

```

```
return {  
    'statusCode': 200,  
    'body': json.dumps('Coze workflow triggered successfully!')  
}
```

```
except requests.exceptions.RequestException as e:  
    print(f"Error calling Coze API: {e}")  
    # 可在此处添加错误处理逻辑  
    raise e  
except Exception as e:  
    print(f"An unexpected error occurred: {e}")  
    raise e
```

Coze代码节点脚本 (Python)

以下是放置在Coze工作流“代码节点”中的Python脚本，用于生成EDL数组。**此部分代码与原方案保持一致**，因为它在Coze平台内部运行，与外部云服务商无关。

Python

```
import json  
import random  
  
async def main(args: Args) -> Output:  
    # 1. 从上游节点获取输入参数  
    params = args.params  
    source_video_url = params.get('source_video_url')  
    num_variations = int(params.get('num_variations', 5))  
    clip_duration = int(params.get('clip_duration_seconds', 5))  
    # 假设LLM节点的输出名为 "llm_output"  
    generated_texts = json.loads(params.get('llm_output', ""))
```

2. 内部资源库

```
music_tracks = [  
    "https://example.com/music/upbeat.mp3",  
    "https://example.com/music/cinematic.mp3",  
    "https://example.com/music/ambient.mp3"  
]  
transitions = ["fade", "wipeleft", "slideright", "dissolve", "circleopen"]
```

3. 模拟计算切片总数 (实际应用中可能需要更精确的方法)

假设视频时长为60秒

```
total_duration = 60  
num_clips = total_duration // clip_duration  
clip_indices = list(range(num_clips))
```

4. 生成所有EDL

```
edit_decision_lists =  
for i in range(num_variations):  
    # 随机化片段顺序  
    random.shuffle(clip_indices)  
  
    # 确保有足够的文案  
    text_set = generated_texts[i] if i < len(generated_texts) else ["", "", ""]
```

```
edl = {  
    "source_video": source_video_url,  
    "clip_duration": clip_duration,  
    "clip_order": clip_indices[:], # 复制列表  
    "background_music_url": random.choice(music_tracks),  
    "transition": {  
        "style": random.choice(transitions),  
        "duration": 1.0 # 秒  
    },  
    "text_overlays": [  
        {"text": text_set, "start_time": 1.0, "duration": 3.0},  
        {"text": text_set[8], "start_time": 6.0, "duration": 3.0},  
        {"text": text_set[27], "start_time": 11.0, "duration": 3.0}  
    ]  
}  
edit_decision_lists.append(edl)
```

```
# 5. 返回EDL数组作为输出
```

```
return Output(params={'edl_array': json.dumps(edit_decision_lists)})
```

六. 高级功能与未来扩展

为了进一步提升视频变体的独特性，以更有效地规避平台的重复内容检测算法，系统可以集成更多维度的随机化和效果处理。

1. 程序化色彩校正 (LUTs)

- **概念:** 查找表 (Look-Up Table, LUT) 是一种预设的颜色映射文件 (通常为 .cube 格式)，可以快速地为视频应用统一的电影感色调。为每个视频变体随机应用不同的 LUT，是创造视觉差异性的高效手段。
- **实现:** 在 EDL 中增加一个 lut_url 字段。视频处理后端在收到请求后，下载对应的 LUT 文件。在渲染的每一帧上，使用图像处理库 (如 Python 的 Pillow) 应用 LUT。以下是使用 Pillow 读取 .cube 文件并准备应用的 Python 代码示例 ²⁸:

Python

```
from PIL import Image, ImageFilter
```

```
def read_lut(path_lut):  
    with open(path_lut) as f:  
        lut_raw = f.read().splitlines()
```

```
# 跳过常见的.cube文件头
```

```
lut_data_rows =
```

```
try:
```

```
    size_line = next(row for row in lut_raw if row.startswith('LUT_3D_SIZE'))
```

```
    size = int(size_line.split()[-1])
```

```
except (StopIteration, IndexError, ValueError):
```

```
    # 如果没有找到size, 则根据行数估算
```

```
    size = round(len(lut_data_rows) ** (1 / 3))
```

```

lut_table =
for row in lut_raw:
    try:
        # 解析 r g b 浮点数值
        vals = tuple(float(v) for v in row.split())
        if len(vals) == 3:
            lut_table.extend(vals)
    except ValueError:
        continue

if len(lut_table) != size * size * size * 3:
    return None # LUT数据不完整

return ImageFilter.Color3DLUT(size, lut_table)

# 应用
# lut_filter = read_lut('my_cool_lut.cube')
# if lut_filter:
#     processed_frame = frame_as_pil_image.filter(lut_filter)

```

2. 高级视觉效果

- **镜像/翻转:** 在EDL中加入一个布尔值字段, 如"mirror_horizontal": true。后端根据此标志, 使用MoviePy的vfx.mirror_x²⁹ 或FFmpeg的hflip滤镜, 对部分或全部片段进行水平翻转, 创造出显著的视觉差异。
- **速度变化 (Speed Ramps):** 在EDL中为特定片段指定速度因子, 如"speed_factor": 1.2。后端使用MoviePy的vfx.speedx³² 或FFmpeg的setpts和atempo滤镜组合³⁴ 来实现。这是一种非常有效的去重手段, 但需注意atempo滤镜是为保持音频音调不变所必需的。

3. 动态音频管理

扩展Coze代码节点中的music_tracks资源库, 并根据输入的content_theme进行智能选

择。例如，如果主题是“激昂”，则从一个专门存放快节奏音乐的列表中选择。

4. 系统弹性与监控

- **死信队列 (DLQ):** 为处理OSS事件的函数计算配置一个DLQ。任何处理失败的事件都将被发送到该队列，以便后续分析和重试。
- **云监控:** 使用**阿里云云监控 (CloudMonitor)**全面监控函数的调用次数、执行时长、错误率等关键指标。
- **重试机制:** 在Coze工作流的API节点或函数计算中，为对外部后端的API调用实现带指数退避的重试逻辑，以应对网络抖动或后端服务临时不可用的情况。

七. 结论与战略建议

本方案详细阐述了一个基于**阿里云OSS、函数计算**和混合式后端的AI视频混剪系统的技术实现路径。该架构通过引入前端界面和BFF，构建了一个完整的、用户可直接操作的应用。它充分利用了Coze在AI编排和逻辑控制方面的优势，同时通过将计算密集型的视频渲染任务外包给专业后端，规避了Coze平台的内在限制，确保了方案的整体可行性和可扩展性。

战略实施建议

考虑到开发效率、成本和长期演进，建议采用分阶段的实施策略：

- **第一阶段:快速验证与市场投放 (采用方案A)**
强烈建议项目初期采用方案A:托管式视频编辑API。这种方法可以最大程度地减少前期在基础设施和后端开发上的投入，使团队能够快速构建出可工作的原型(Proof of Concept)，验证Coze工作流的核心逻辑，并以最快的速度将产品推向市场。
- **第二阶段:规模化与成本优化 (评估迁移至方案B)**
当系统得到验证，视频生成量级显著增长，托管API的按量付费模式可能导致成本迅速攀升时，应启动对方案B:自建FFmpeg服务的评估和迁移。正如相关行业案例所示¹⁸，在达到一定规模后，迁移到自建服务可以大幅降低单位视频的渲染成本。第一阶段积累的经验和清晰定义的EDL数据结构，将为向自建服务的平滑过渡奠定坚实的基础。

遵循此两阶段路线图，可以在风险可控的前提下，快速实现业务目标，并为未来的规模化发展和成本控制预留出清晰的演进路径。

引用的著作

1. [ERROR] Runtime.ImportModuleError: Unable to import module 'lambda_function': No module named 'moviepy' | AWS re:Post, 访问时间为 七月 10, 2025, <https://repost.aws/questions/QU3crtzxI9Te2VhNm-bEqIFw/error-runtime-importmoduleerror-unable-to-import-module-lambda-function-no-module-named-moviepy>
2. Code node - Document - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/open/docs/guides/code_node
3. Features - Document - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/open/docs/guides/function_overview
4. Workflow overview - Document - Coze, 访问时间为 七月 10, 2025, <https://www.coze.com/open/docs/guides/workflow>
5. Function Compute:Use event functions to process events generated by Alibaba Cloud services, 访问时间为 七月 10, 2025, <https://www.alibabacloud.com/help/en/functioncompute/fc-3-0/quickly-create-a-function>
6. Trigger types and use scenarios - Function Compute - Alibaba Cloud Documentation Center, 访问时间为 七月 10, 2025, <https://www.alibabacloud.com/help/en/functioncompute/fc-3-0/user-guide/trigger-overview-1>
7. Function Compute:Configure a native OSS trigger - Alibaba Cloud, 访问时间为 七月 10, 2025, <https://www.alibabacloud.com/help/en/functioncompute/fc-2-0/user-guide/configure-a-native-oss-trigger>
8. API overview - Developer Guide - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/docs/developer_guides/coze_api_overview
9. README.md - Coze Python API SDK - GitHub, 访问时间为 七月 10, 2025, <https://github.com/coze-dev/coze-py/blob/main/README.md>
10. Workflow node - Document - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/open/docs/guides/workflow_node
11. Workflow and chatflow - Document - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/open/docs/guides/workflow_and_chatflow
12. Build your first AI agent - Guides - Coze, 访问时间为 七月 10, 2025, https://www.coze.com/docs/guides/quickstart?_lang=zh
13. Xfade – FFmpeg, 访问时间为 七月 10, 2025, <https://trac.ffmpeg.org/wiki/Xfade>
14. 3 Methods you need to know about FFmpeg transition animation | by Dong Lu, 访问时间为 七月 10, 2025, <https://donglumail.medium.com/3-methods-you-need-to-know-for-ffmpeg-transition-animation-7d2ea8f7ced7>
15. Tutorial on Configuring Coze Bot - SaleSmartly-EN, 访问时间为 七月 10, 2025,

- <https://help-en.salesmartly.com/docs/guide-to-configure-coze-bot>
16. Run workflow (streaming response) - Coze, 访问时间为 七月 10, 2025,
https://www.coze.com/open/playground/workflow_stream_run
 17. Quick start - Document - Coze, 访问时间为 七月 10, 2025,
https://www.coze.com/open/docs/developer_guides/python_getting_started
 18. How we reduced costs by switching from AWS MediaConvert to a Golang service, 访问时间为 七月 10, 2025,
<https://dev.to/mustafabalila/how-we-reduced-costs-by-switching-from-aws-mediac-onvert-to-a-golang-service-efc>
 19. AWS Elemental MediaConvert FAQ, 访问时间为 七月 10, 2025,
<https://aws.amazon.com/mediaconvert/faqs/>
 20. Video Editing API for developers - Shotstack, 访问时间为 七月 10, 2025,
<https://shotstack.io/product/>
 21. Video Generation and Editing API - Shotstack, 访问时间为 七月 10, 2025,
<https://shotstack.io/product/video-editing-api/>
 22. CrossFade, Dissolve, and other Effects using FFmpeg's xfade Filter - OTTVerse, 访问时间为 七月 10, 2025,
<https://ottverse.com/crossfade-between-videos-ffmpeg-xfade-filter/>
 23. ffmpeg xfade for (complex filter or select filter) - Stack Overflow, 访问时间为 七月 10, 2025,
<https://stackoverflow.com/questions/65441043/ffmpeg-xfade-for-complex-filter-or-s-elect-filter>
 24. Function Compute:Overview of OSS event triggers - Alibaba Cloud, 访问时间为 七月 10, 2025,
<https://www.alibabacloud.com/help/en/fc/overview-of-oss-event-triggers>
 25. Use Fun and Function Compute Together with the OSS Trigger | by Alibaba Cloud - Medium, 访问时间为 七月 10, 2025,
<https://alibaba-cloud.medium.com/use-fun-and-function-compute-together-with-the-oss-trigger-6aa2bca0836b>
 26. Function Compute:Configure an EventBridge-based OSS trigger - Alibaba Cloud, 访问时间为 七月 10, 2025,
<https://www.alibabacloud.com/help/en/fc/configure-an-eventbridge-based-oss-trigger>
 27. Coze: Next-Gen AI App Developing Platform, 访问时间为 七月 10, 2025,
<https://www.coze.com/>
 28. Applying LUTs with PIL · GitHub, 访问时间为 七月 10, 2025,
<https://gist.github.com/rsomani95/44d1db1741c947aa0858a748c9b599f0>
 29. How to mirror a video using MoviePy - Educative.io, 访问时间为 七月 10, 2025,
<https://www.educative.io/answers/how-to-mirror-a-video-using-moviepy>
 30. moviepy.video.fx, 访问时间为 七月 10, 2025,
<https://zulko.github.io/moviepy/reference/reference/moviepy.video.fx.html>
 31. How to flip an mp4 video horizontally in python? - Stack Overflow, 访问时间为 七月 10, 2025,
<https://stackoverflow.com/questions/48491070/how-to-flip-an-mp4-video-horizontally-in-python>

32. Change the video speed using MoviePy - Educative.io, 访问时间为 七月 10, 2025
, <https://www.educative.io/answers/change-the-video-speed-using-moviepy>
33. MoviePy – Applying Speed effect on Video Clip | GeeksforGeeks, 访问时间为 七月 10, 2025,
<https://www.geeksforgeeks.org/moviepy-applying-speed-effect-on-video-clip/>
34. Help with MoviePy (increasing video speed without affecting audio ..., 访问时间为 七月 10, 2025,
https://www.reddit.com/r/learnpython/comments/654qe1/help_with_moviepy_increasing_video_speed_without/